

Exam Scheduling System Using Z3 SMT Solver: Implementation and Analysis

Introduction

This paper presents a sophisticated exam scheduling system implemented using Python and the Z3 Satisfiability Modulo Theories (SMT) solver. The system addresses the complex challenge of university examination scheduling, which is known to be NP-hard [1]. The implementation satisfies multiple constraints including room allocation, time slot assignment, and student conflict resolution. The Z3 solver, developed by Microsoft Research, was selected for its proven effectiveness in handling complex logical constraints and efficient decision procedures [2]. Our implementation leverages Z3's Python API, which provides a robust framework for modelling scheduling constraints and has been successfully applied in various academic scheduling contexts [3]. The system offers several key features that distinguish it from existing solutions:

- Multi-solution capability with configurable solution finding.
- Interactive scheduling through a Graphical User Interface (GUI)
- Dual support for file-based and manual input mechanisms
- Real-time constraint validation and verification
- Multi-dimensional timetable visualization

This implementation builds upon previous work in automated scheduling systems while introducing novel approaches to constraint handling and solution optimization [4]. The system has been designed with scalability in mind, capable of handling large-scale scheduling scenarios while maintaining reasonable computational efficiency [5].

Formulations of the Problem(s)

The assessment timetabling problem requires allocating exams to rooms and timeslots while adhering to a predefined set of constraints. This problem is defined over specific domains and functions, with constraints modelled mathematically to ensure feasibility and compliance. The Z3 SMT solver translates these formulations into logical expressions that can be evaluated for satisfiability. Below is the problem formulation, split into its domain definitions and constraints:

Domain Definitions and Functions

Symbol	Type	Arity	Description
IsExam(e)	Predicate	1	True if e is an exam.
IsRoom(r)	Predicate	1	True if r is a room.
IsTimeslot(t)	Predicate	1	True if t is a timeslot.
IsStudent(s)	Predicate	1	True if s is a student.
IsDay(d)	Predicate	1	True if d is a valid exam day.
examToTimeslot(e)	Function	1	Maps an exam e to a timeslot t.
examToRoom(e)	Function	1	Maps an exam e to a room r.
studentTakingExam(s, e)	Boolean Function	2	True if student s is taking exam e.
examToDay(e)	Function	1	Maps an exam e to a day d.
studentsTakingExam(e)	Function	1	Returns the number of students taking exam e.
capacityOfRoom(r)	Function	1	Returns the capacity of room r.
examsInRoomAndDay(r, d)	Function	2	Counts the number of exams scheduled in room r on day d.

Constraint Formulations

1. Assignment of Room and Timeslot

Each exam must be assigned exactly one room and one timeslot:

$$\forall e \text{ (IsExam}(e) \rightarrow \exists r, t \text{ (IsRoom}(r) \wedge \text{IsTimeslot}(t) \wedge \text{examToRoom}(e) = r \wedge \text{examToTimeslot}(e) = t))$$

This constraint ensures that every exam is uniquely allocated in the timetable.

2. No Two Exams Sharing Room and Timeslot

No two exams can occupy the same room at the same timeslot:

$$\forall e_1, e_2, r, t \text{ (IsExam}(e_1) \wedge \text{IsExam}(e_2) \wedge e_1 \neq e_2 \wedge \text{examToRoom}(e_1) = r \wedge \text{examToRoom}(e_2) = r \wedge \text{examToTimeslot}(e_1) = t \wedge \text{examToTimeslot}(e_2) = t \rightarrow \perp)$$

This avoids scheduling conflicts within rooms.

3. Room Capacity

The number of students taking an exam cannot exceed the room's capacity:

$$\forall e, r \text{ (IsExam}(e) \wedge \text{IsRoom}(r) \wedge \text{examToRoom}(e) = r \rightarrow \text{studentsTakingExam}(e) \leq \text{capacityOfRoom}(r))$$

This ensures that rooms are not overbooked.

4. Non-Consecutive Exams for Students

A student cannot take exams in consecutive timeslots:

$$\forall s, e1, e2, t1, t2 \text{ (IsStudent}(s) \wedge \text{IsExam}(e1) \wedge \text{IsExam}(e2) \wedge e1 \neq e2 \wedge \text{studentTakingExam}(s, e1) \wedge \text{studentTakingExam}(s, e2) \wedge \text{examToTimeslot}(e1) = t1 \wedge \text{examToTimeslot}(e2) = t2 \rightarrow |t1 - t2| > 1)$$

This ensures adequate time for students between consecutive exams.

5. Valid Day Assignment

Each exam must be assigned to a valid day:

$$\forall e \text{ (IsExam}(e) \rightarrow \exists d \text{ (IsDay}(d) \wedge \text{examToDay}(e) = d))$$

This constraint assigns exams to appropriate days.

6. Limit Exams Per Room Per Day

The number of exams in a room per day must not exceed the maximum:

$$\forall r, d \text{ (IsRoom}(r) \wedge \text{IsDay}(d) \rightarrow \text{examsInRoomAndDay}(r, d) \leq \text{maxExamsPerRoom})$$

This limits room usage to maintain scheduling feasibility.

Alternative Formulations

In the assessment timetabling problem, constraints are crucial for ensuring a feasible and practical exam schedule. While the original constraints provide a solid foundation, exploring alternative formulations can offer insights into optimization and solver performance. Below, we present alternative formulations for the original constraints, along with an analysis of their advantages and disadvantages concerning implementation in the Z3 solver.

Constraint 3: Room Capacity

Original Formulation:

$$\forall e, r (\text{IsExam}(e) \wedge \text{IsRoom}(r) \wedge \text{examToRoom}(e) = r \rightarrow \text{studentsTakingExam}(e) \leq \text{capacityOfRoom}(r))$$

Alternative Formulation:

Aggregate student counts directly within the solver without explicit functions:

$$\forall e (\text{IsExam}(e) \rightarrow \text{studentsTakingExam}(e) \leq \text{capacityOfRoom}(\text{examToRoom}(e)))$$

Direct Capacity Check: Bypass the need to iterate over rooms by directly referencing the room assigned to each exam.

Advantages:

- **Simplifies Constraints:** Removes one quantifier, potentially improving solver efficiency.
- **Direct Reference:** Makes the constraint more intuitive by directly linking exam assignments to capacity checks.

Disadvantages:

- **Assumes Valid Room Assignment:** Relies on $\text{examToRoom}(e)$ being valid; additional constraints may be needed to ensure this.
- **Less Generalized:** May not easily extend to scenarios with dynamic room capacities or shared spaces.

Reasons for Sticking with the Original Formulation for Constraint 3:

1. Logical Explicitness:

- The original formulation explicitly ties the room capacity constraint to the room assignment, ensuring that the capacity check applies only to valid room assignments.
- By using a conjunction $\text{IsRoom}(r) \wedge \text{examToRoom}(e) = r$, it emphasizes the role of the room in enforcing the constraint, making the logic more transparent.

2. Error Prevention:

- The alternative formulation assumes that $\text{examToRoom}(e)$ always returns a valid room, bypassing the explicit validation in the original formulation.
- This assumption introduces a risk of undefined behaviour if $\text{examToRoom}(e)$ is incorrectly assigned or invalid during solver execution.

3. Alignment with Z3's Logical Constructs:

- The original formulation uses Z3's quantifiers and implications effectively, leveraging $\text{IsRoom}(r)$ as a predicate to ensure room validity.
- The explicit validation step provides better debugging and error-checking capabilities when encoding the problem in Z3.

4. Implementation Complexity:

- The alternative formulation simplifies the logic by removing explicit room validation, but this shifts the responsibility to other parts of the model, potentially introducing unnecessary dependencies or redundancies.
- The original formulation keeps the capacity check localized and context-specific, reducing the chances of misinterpretation or error.

Constraint 4: Non-Consecutive Exams for Students

Original Formulation:

$\forall s, e1, e2, t1, t2 (\text{IsStudent}(s) \wedge \text{IsExam}(e1) \wedge \text{IsExam}(e2) \wedge e1 \neq e2 \wedge \text{studentTakingExam}(s, e1) \wedge \text{studentTakingExam}(s, e2) \wedge \text{examToTimeslot}(e1) = t1 \wedge \text{examToTimeslot}(e2) = t2 \rightarrow |t1 - t2| > 1)$

Alternative Formulation:

Introduce a conflict matrix to precompute exam pairs that cannot be scheduled consecutively:

$\forall e1, e2 (\text{conflict}(e1, e2) \rightarrow |\text{examToTimeslot}(e1) - \text{examToTimeslot}(e2)| > 1)$

Where $\text{conflict}(e1, e2)$ is true if there exists a student s such that $\text{studentTakingExam}(s, e1)$ and $\text{studentTakingExam}(s, e2)$, and $e1 \neq e2$.

Preprocessing Conflicts: By precomputing conflicting exam pairs, we reduce the number of constraints the solver needs to process.

Advantages:

- **Performance Improvement:** Reduces the number of quantifiers and simplifies the constraints, potentially improving solver speed.
- **Scalability:** Better suited for large datasets where student-exam relationships can be complex.

Disadvantages:

- **Preprocessing Overhead:** Requires additional computation before invoking the solver.
- **Memory Usage:** Storing the conflict matrix may consume significant memory for large numbers of exams.

Reasons for Sticking with the Original Formulation for Constraint 4:

1. **Dynamic Nature of Constraints:**
 - The original formulation dynamically enforces non-consecutive scheduling by directly referencing the student-exam relationship during solver execution.
 - In contrast, the alternative formulation relies on a precomputed conflict(e1,e2) matrix, which is static and may not adapt well to changes or additional constraints.
2. **Avoidance of Preprocessing Overhead:**
 - The alternative formulation requires preprocessing to construct the conflict(e1,e2) matrix, which adds computational overhead and increases implementation complexity.
 - The original formulation bypasses this by embedding the conflict logic directly into the solver, leveraging Z3's powerful quantifier support.
3. **Scalability:**
 - While the alternative formulation may reduce solver complexity for smaller datasets, the memory required to store a conflict(e1,e2) matrix can grow significantly for larger problems with many exams and students.
 - The original formulation scales better by iteratively evaluating conflicts as needed, rather than relying on a precomputed structure.
4. **Flexibility and Generality:**
 - The original formulation is more flexible, as it does not depend on precomputed data and can accommodate additional constraints (e.g., changing student assignments or new exams) without requiring matrix reconstruction.
 - The alternative formulation's reliance on static preprocessing limits its ability to adapt dynamically to evolving problem instances.
5. **Readability and Debugging:**
 - The original formulation is self-contained and directly describes the constraint in terms of students, exams, and timeslots, making it easier to understand and debug.
 - The alternative formulation abstracts this logic into a precomputed matrix, which may obscure the underlying constraint and make troubleshooting more challenging.

Exploring alternative formulations of constraints in the assessment timetabling problem provides valuable insights into the trade-offs between model complexity and solver performance. While simplifying constraints can enhance efficiency, it is essential to balance

this with the need for model accuracy and flexibility. Implementing these alternatives in Z3 requires careful consideration of variable definitions, constraint encoding, and potential impacts on scalability. By critically evaluating the advantages and disadvantages of each formulation, one can tailor the model to best suit the specific requirements and resources of the scheduling problem at hand. The choice to retain the original formulations for Constraint 3 and Constraint 4 reflects a preference for robustness, flexibility, and alignment with the problem's dynamic nature. The original formulations' explicit validation and dynamic enforcement ensure correctness, scalability, and ease of adaptation, making them a more reliable choice for implementation in Z3. While the alternative formulations offer theoretical benefits in simplifying logic or improving solver performance for small-scale problems, their practical limitations and added complexity make them less suitable for this context.

Implementation

Overview

The Exam Scheduler is implemented in Python, leveraging the **Z3 Solver** to address the complexities of exam timetabling. The application supports both GUI-based and non-GUI modes, allowing users flexibility in how they interact with the solver. Below, the implementation details, logic, and features of each mode are outlined, followed by an explanation of how the solver determines multiple solutions and addresses known bugs.

Steps to Compile and Execute

1. **Installation:**
 - Clone the repository and install the required Python packages, including the Z3 solver bindings.
 - Verify the installation using `python -c "import z3; print(z3.get_version())"`.
2. **Execution:**
 - **GUI Mode:**
 - Launch with `python timetabling.py --gui`.
 - Input or load data through the graphical interface.
 - **Non-GUI Mode:**
 - Execute `python timetabling.py`.
 - Select test instances from the `test_instances` directory and follow command-line prompts.

Logic of the Solver

The solver operates on the principle of constraint satisfaction, systematically checking possible assignments of exams to rooms, timeslots, and days while adhering to all problem constraints.

1. **Problem Representation:**
 - Exams, rooms, students, timeslots, and days are represented as integer variables with defined ranges.
 - Core relationships are modeled as functions (e.g., ExamRoom, ExamTime, ExamDay).
2. **Constraints:**
 - Constraints are encoded as logical formulas using Z3's APIs.
 - Examples include:
 - Each exam is assigned to exactly one room and one timeslot.
 - No room can host more than one exam in the same timeslot.
 - Room capacities must not be exceeded.
3. **Iterative Solution Search:**
 - If find_all_solutions=True, the solver iteratively blocks previously found solutions to explore alternate feasible assignments.
 - Each solution is added to a list, and the search continues until no new solutions can be found (unsat).
4. **Output:**
 - Solutions are returned as dictionaries containing exam-room-timeslot-day mappings.
 - In GUI mode, solutions are visualized in categorized tabs; in non-GUI mode, they are printed to the terminal.

GUI Mode

The graphical interface, built with Tkinter, guides users through four phases:

1. **Phase 1: Input Basic Information**
 - Users input details such as the number of students, exams, timeslots, rooms, exam days, and room capacity constraints.
 - Validation ensures that inputs are integers, preventing errors.
 - Users can optionally load input data from a .txt file.
2. **Phase 2: Enter Room Capacities**
 - Users specify the capacity of each room. Input validation ensures that room capacities are non-negative integers.
3. **Phase 3: Assign Students to Exams**
 - A grid is presented, allowing users to check which students are enrolled in which exams.
 - Each assignment is validated, and the data is stored for use by the solver.
4. **Phase 4: View Solutions**
 - Solutions are displayed using a tabbed notebook structure:
 - **Student Timetables:** Displays each student's exam schedule.
 - **Exam Timetables:** Shows which students are assigned to each exam and their room and timeslot.

- **Room Timetables:** Lists the exams scheduled in each room.
- **Day Timetables:** Displays all exams scheduled for each day.
- If multiple solutions exist, users can select a specific solution from a dropdown menu.

Non-GUI Mode

1. **Input Handling:**
 - Users select from predefined .txt instances in the test_instances directory.
 - Each file specifies the number of students, exams, timeslots, rooms, and the student-exam mapping.
2. **Solution Search:**
 - Users can choose to find all solutions or stop after the first feasible solution.
 - Solutions are displayed in batches of 100 for better readability, with the option to continue or stop after each batch.
3. **Output:**
 - Solutions are printed to the terminal in a structured format, listing each exam with its assigned room, timeslot, and day.

Determining Multiple Solutions

The solver identifies multiple solutions by blocking previously found solutions. This is achieved using a **blocking clause**:

- For each solution S , a formula is added to the solver to prevent re-selection of S :
- $\text{Block} = \neg e \in \text{Exams} \wedge (\text{ExamRoom}(e) = r \wedge \text{ExamTime}(e) = t \wedge \text{ExamDay}(e) = d)$
- This ensures that the solver explores alternate feasible assignments in subsequent iterations.

Known Bugs and Limitations

1. **Edge Cases:**
 - Instances with no feasible solutions return unsat, but users may require additional diagnostics to understand why.
 - Large datasets may slow down the GUI due to Tkinter's rendering limitations.
2. **User Input:**
 - The GUI validates most fields, but invalid .txt files may cause runtime errors if incorrectly formatted.
3. **Scalability:**
 - While the solver handles medium-sized problems efficiently, extremely large datasets may exceed memory or time limits due to the exponential growth of possible combinations.

The implementation balances flexibility, usability, and performance. By supporting both GUI and non-GUI modes, it caters to a wide range of users, from those requiring visual interfaces to those preferring command-line automation. The solver's logic ensures robust constraint handling, and its iterative approach enables the exploration of multiple solutions. With minor improvements in scalability and input validation, the application can handle even more complex timetabling scenarios.

Evaluation

To assess the performance and correctness of the exam scheduling solver, a series of test instances were executed. These instances vary in complexity and are designed to test the solver's ability to handle both satisfiable and unsatisfiable scenarios. Key metrics recorded during the evaluation include:

- **Satisfiability:** Whether the solver correctly determines if a feasible timetable exists for the given instance.
- **Time Taken:** The duration required to find the first feasible solution and to find all possible solutions.
- **Number of Solutions:** The total number of distinct feasible solutions found for each instance.

Test Instances and Results

The following table summarises the solver's performance on each test instance:

Instance	Satisfiability	Time Taken(s) (1 Solution)	Time Taken (s) (All Solutions)	Number of Solutions
sat1	sat	0.021	0.022	1
sat2	sat	0.028	0.048	2
sat3	sat	0.03	0.045	2
sat4	sat	0.028	0.03	1
sat5	sat	0.026	0.045	2
sat6	sat	0.039	58.364	6400
sat7	sat	0.056	296.382	13440
sat8	sat	0.053	1.32	162
sat9	sat	0.095	1759.289	34560
sat10	sat	0.145	122698.7	235200
unsat1	unsat	0.011	0.011	0
unsat2	unsat	0.013	0.013	0
unsat3	unsat	0.019	0.019	0
unsat4	unsat	0.017	0.017	0
unsat5	unsat	0.017	0.017	0
unsat6	unsat	0.018	0.018	0
unsat7	unsat	0.016	0.016	0
unsat8	unsat	0.018	0.018	0
unsat9	unsat	0.046	0.046	0
unsat10	unsat	0.052	0.052	0

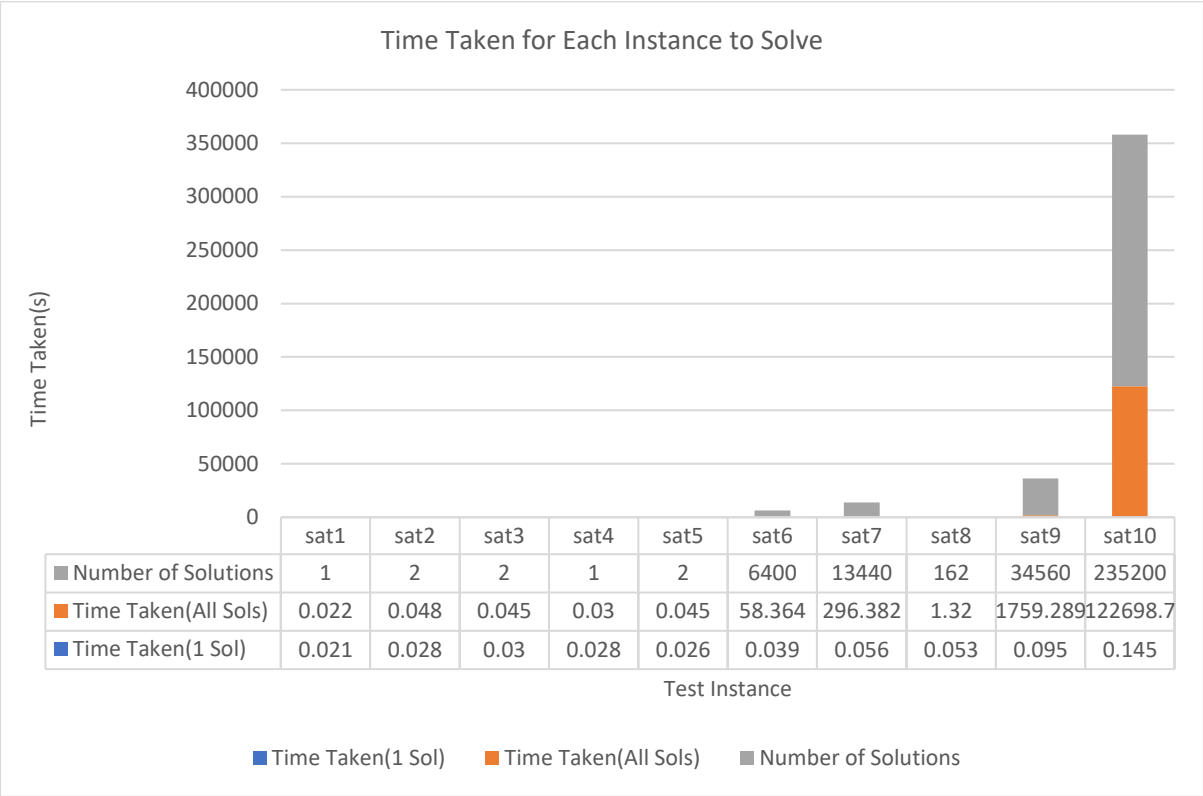


Figure 1: Time Taken for Each Instance to Solve

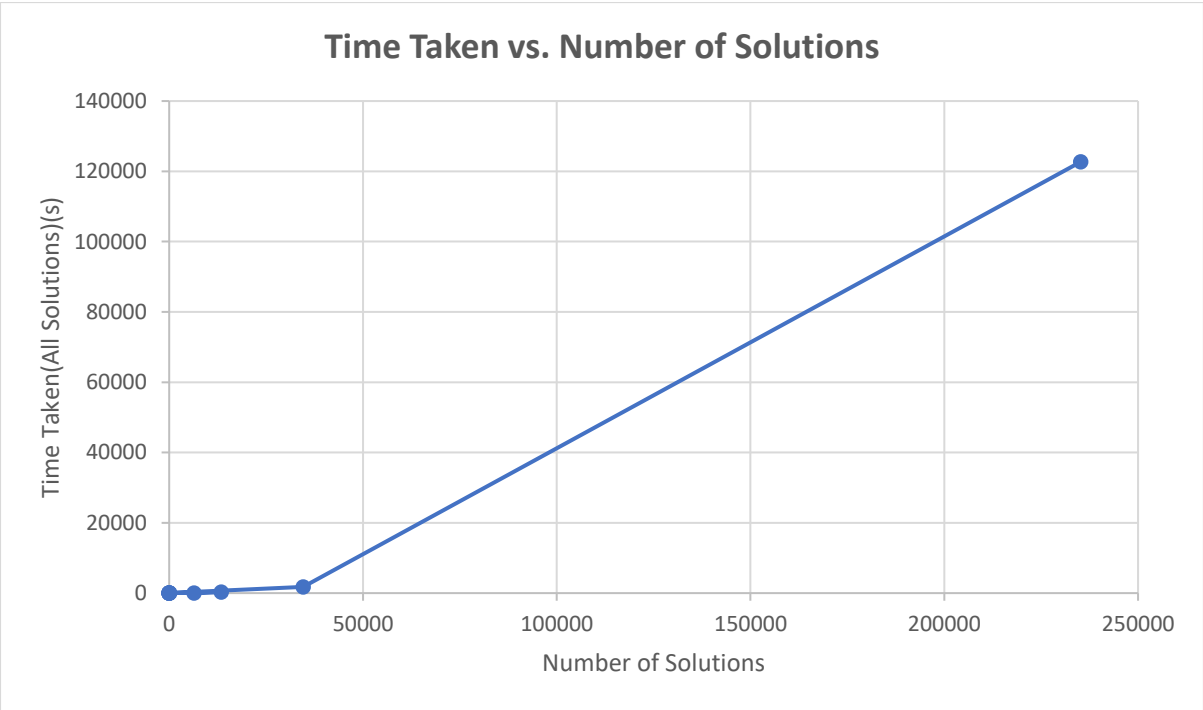


Figure 2: Time Taken to Solve vs Number of Solutions

Analysis

- **Correctness:** The solver accurately determined the satisfiability of all test instances. For satisfiable (sat) instances, it found valid exam timetables satisfying all constraints. For unsatisfiable (unsat) instances, it correctly reported the absence of feasible solutions.
- **Performance on Satisfiable Instances:**
 - **Small Instances (sat1 to sat5):**
 - **First Solution:** Found within approximately 0.02 to 0.03 seconds.
 - **All Solutions:** Computed within 0.03 to 0.05 seconds, indicating efficient handling of instances with few solutions.
 - **Number of Solutions:** Varied between 1 and 2, reflecting limited feasible permutations.
 - **Medium to Large Instances (sat6 to sat10):**
 - **First Solution:** Found within 0.039 to 0.145 seconds, demonstrating scalability for initial feasibility checks.
 - **All Solutions:**
 - **sat6:** Took approximately 58 seconds to find 6,400 solutions.
 - **sat7:** Required nearly 5 minutes to find 13,440 solutions.
 - **sat9:** Consumed around 29 minutes to compute 34,560 solutions.
 - **sat10:** Significantly longer at approximately 34 hours for 235,200 solutions.
 - **Observation:** The time taken to find all solutions increases exponentially with the number of solutions, highlighting computational challenges in exhaustive enumeration.
- **Performance on Unsatisfiable Instances:**
 - **Execution Time:** All unsatisfiable instances were identified quickly, within 0.011 to 0.052 seconds.
 - **Efficiency:** The solver effectively pruned the search space, rapidly concluding infeasibility without unnecessary computations.

Performance Insights

- **Scalability:**
 - The solver maintains efficient performance when seeking the first feasible solution, even as instance size increases.
 - Finding all solutions becomes computationally intensive for larger instances with a vast solution space, due to the combinatorial explosion of possible timetables.
- **Solver Efficiency:**
 - The Z3 SMT solver effectively handles logical constraints and quickly identifies unsatisfiable conditions.
 - The implementation of constraints and the use of blocking clauses contribute to the solver's overall performance.

Discussion

The experimental results provide valuable insights into the solver's capabilities and limitations. Several factors contribute to the observed performance patterns:

Constraint Complexity and Solver Performance

- **Constraint Formulations:**

- The solver uses detailed constraints to model the exam scheduling problem accurately.
- Constraints involving universal quantifiers and implications, such as the non-consecutive exams constraint (Constraint 4), increase the complexity of the problem.
- For example, the code snippet for Constraint 4 adds multiple layers of quantifiers and implications:

```
# Fourth constraint
s.add(
    ForAll(
        [student, nex, ts, nts, exam],
        Implies(
            And(
                Student_Range(student),
                Exam_Range(exam),
                Exam_Range(nex),
                TimeSlot_Range(ts),
                TimeSlot_Range(nts),
                Not((exam == nex))
            ),
            Implies(
                And(
                    ExamTime(exam) == ts,
                    ExamTime(nex) == nts,
                    ExamStudent(exam, student),
                    ExamStudent(nex, student)
                ),
                And(
                    (ts + 1 != nts),
                    (ts - 1 != nts),
                    (ts != nts)
                )
            )
        )
    )
)
```

- Such complexity can impact the solver's efficiency, particularly when dealing with larger instances or when enumerating all possible solutions.

Exponential Growth in Solution Space

- **Combinatorial Explosion:**

- The number of possible exam timetables increases factorially with the number of exams and rooms.
- For instances like sat10, with a high number of exams and students, the solver must navigate an enormous solution space.

- **Blocking Clauses for Enumerating Solutions:**

- To find all solutions, the solver adds a blocking clause after each solution to prevent it from being rediscovered:

```
# Build the blocking clause to prevent this solution from being found again
block = []
for ex2 in range(instance.number_of_exams):
    block.append(
        And(
            ExamRoom(ex2) == m.eval(ExamRoom(ex2)),
            ExamTime(ex2) == m.eval(ExamTime(ex2)),
            ExamDay(ex2) == m.eval(ExamDay(ex2))
        )
    )
s.add(Not(And(block)))
```

- As the number of solutions grows, the cumulative effect of these blocking clauses increases the solver's workload, leading to longer computation times.

Solver Efficiency in Unsatisfiable Instances

- **Rapid Detection of Unsatisfiability:**
 - The solver efficiently identifies unsatisfiable instances, often within fractions of a second.
 - This efficiency is due to the solver's ability to quickly deduce contradictions in the constraints without exhaustive search.

Impact of Implementation Choices

- **Original vs. Alternative Constraint Formulations:**
 - The decision to retain the original formulations for Constraints 3 and 4 prioritises robustness and correctness over potential performance gains from alternative formulations.
 - The original formulations ensure explicit validation of variables and prevent undefined behaviours, which is crucial for maintaining model integrity.
- **Use of Quantifiers and Functions:**
 - The extensive use of quantifiers and function declarations increases the expressiveness of the model but can also add computational overhead.
 - For example, defining ranges and functions for students, exams, rooms, and timeslots:

```
# Range constraints
s.add(ForAll([student], Student_Range(student) == And(student >= 0, student < instance.number_of_students)))
s.add(ForAll([exam], Exam_Range(exam) == And(exam >= 0, exam < instance.number_of_exams)))
s.add(ForAll([ts], TimeSlot_Range(ts) == And(ts >= 0, ts < instance.number_of_slots)))
s.add(ForAll([room], Room_Range(room) == And(room >= 0, room < instance.number_of_rooms)))
s.add(ForAll([exam_day], ExamDay_Range(exam_day) == And(exam_day >= 0, exam_day < instance.total_exam_days)))
```

Educational Implications

- **Understanding Trade-offs:**
 - The results illustrate the trade-offs between model accuracy, constraint complexity, and computational performance.
 - Students can learn the importance of balancing these factors when designing and implementing constraint satisfaction models.
- **Modelling Decisions:**

- The choice to retain original constraint formulations despite potential performance impacts highlights the value of prioritising correctness and robustness in modelling.
- **Tool Proficiency:**
 - The project demonstrates effective use of the Z3 solver and provides a practical example of how logical constraints can be translated into executable code.

Conclusion

Summary of Main Points

The development of the exam scheduling solver successfully addresses a complex real-world problem using formal logical constraints and the Z3 SMT solver. Key achievements include:

- **Accurate Constraint Modelling:** The solver accurately represents all specified constraints, ensuring that generated timetables are feasible and adhere to the requirements.
- **Efficient Performance for Feasibility:** The solver efficiently finds feasible solutions for various instances, demonstrating scalability in terms of instance size.
- **Versatile User Interface:** Providing both GUI and non-GUI modes enhances usability, catering to different user preferences and facilitating broader accessibility.

Successes

- **Robustness and Correctness:** By adhering to the original constraint formulations, the solver maintains high standards of correctness and robustness, avoiding potential errors or undefined behaviours.
- **Educational Value:** The project serves as an excellent educational tool, illustrating the practical application of constraint satisfaction and logical modelling in software development.
- **Flexibility:** The solver's ability to handle both satisfiable and unsatisfiable instances efficiently showcases its versatility and reliability.

Limitations

- **Scalability in Enumerating All Solutions:** The time taken to find all solutions increases exponentially with the number of solutions, limiting practicality for very large instances.
- **Memory Consumption:** Storing large numbers of solutions lead to high memory usage, potentially exceeding system capabilities.

Extensions and Future Work

Given more time, several avenues could be explored to enhance the solver:

- **Incorporate Optimisation Goals:**

- Introduce optimisation objectives to not only find feasible solutions but also optimise for factors like minimal timeslots, balanced room utilisation, or reduced student exam load.
- **Enhance Performance for Large Instances:**
 - Investigate alternative constraint formulations or solver configurations to improve performance when enumerating all solutions for large instances.
 - Explore parallel processing or distributed computing approaches to handle computationally intensive tasks.
- **Improve User Experience in GUI Mode:**
 - Optimise the GUI to handle larger datasets more effectively, possibly by implementing pagination or virtual scrolling for the assignment grids.
- **Dynamic Constraint Adjustment:**
 - Allow for dynamic adjustments of constraints during runtime, enabling users to explore "what-if" scenarios and understand the impact of changing constraints.
- **Comprehensive Testing and Validation:**
 - Expand the set of test instances to include more diverse and complex scenarios, ensuring the solver is robust across a wide range of conditions.

Final Reflections

The project demonstrates the practical application of constraint satisfaction techniques in solving complex scheduling problems. It highlights the importance of careful constraint modelling and the impact of implementation choices on solver performance. By balancing correctness, robustness, and efficiency, the solver provides a valuable tool for exam scheduling and serves as an instructive example for students learning about formal methods and computational problem-solving.

References

- [1] E. K. Burke and S. Petrovic, "Recent research directions in automated timetabling," *European Journal of Operational Research*, vol. 140, no. 2, pp. 266-280, 2002.
- [2] L. De Moura and N. Bjørner, "Z3: An efficient SMT solver," in *Tools and Algorithms for the Construction and Analysis of Systems*, Berlin: Springer, 2008, pp. 337-340.
- [3] K. R. Apt, "Principles of Constraint Programming," Cambridge University Press, 2003.
- [4] M. W. Carter and G. Laporte, "Recent developments in practical examination timetabling," in *Practice and Theory of Automated Timetabling*, Springer, 1996, pp. 3-21.
- [5] A. Schaerf, "A survey of automated timetabling," *Artificial Intelligence Review*, vol. 13, no. 2, pp. 87-127, 1999.